

Data Science

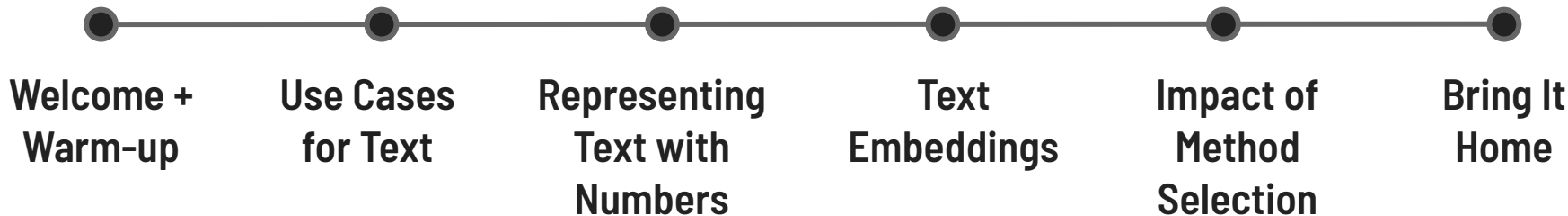
Foundations of Modern NLP

NLP and Generative AI



GENERAL ASSEMBLY

Lesson roadmap



Learning objectives



1

Define common use cases for text data.

2

Describe the evolution of text representation from traditional methods to embeddings.

3

Apply CountVectorizer, TF-IDF to complete a sentiment analysis of customer reviews.

4

Compare two embedding methods, Word2Vec and BERT, using customer reviews.

5

Discuss real-word considerations for embedding selection.

A black and white photograph of a train station platform. Several people are walking away from the camera towards a bright light source at the end of the platform, creating long silhouettes. The floor is wet and reflective. On the left side of the image, there is a solid red vertical bar. Overlaid on this bar is a white rectangular box containing the text 'Warm-Up' in a large, bold, black sans-serif font.

Warm-Up

Working with Text

How would you like to work with text data?

- ☐ Think about the types of data you work with that contain text.
- ☐ Describe the types of tasks you would like to use text data to accomplish.
- ☐ Brainstorm potential uses if you had a powerful AI system helping you parse through the data.

What is Natural Language?

*“To understand text analytics and natural language processing, we need to understand what makes a language “natural.” In simple terms, **natural language** is one developed and evolved by humans through natural use and communication, rather than constructed and created artificially, like a computer programming language.”*

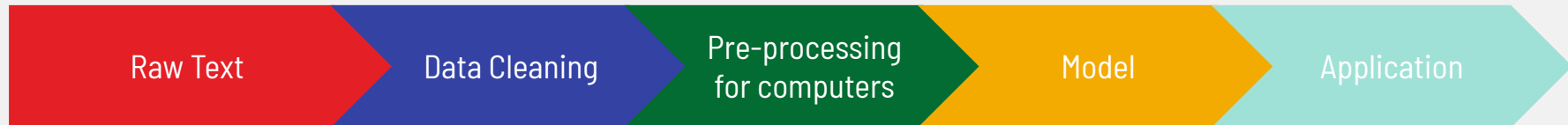
- Dipanjan Sarkar, *Text Analytics with Python: A Practitioner's Guide to NLP*

What is Natural Language Processing?

Natural Language Processing (NLP) is a specialized field of computer science primarily concerned with designing and building applications and systems for facilitating the interaction between machines and the natural language of humans.

[Definition paraphrased from Sarkar.]

NLP General Workflow

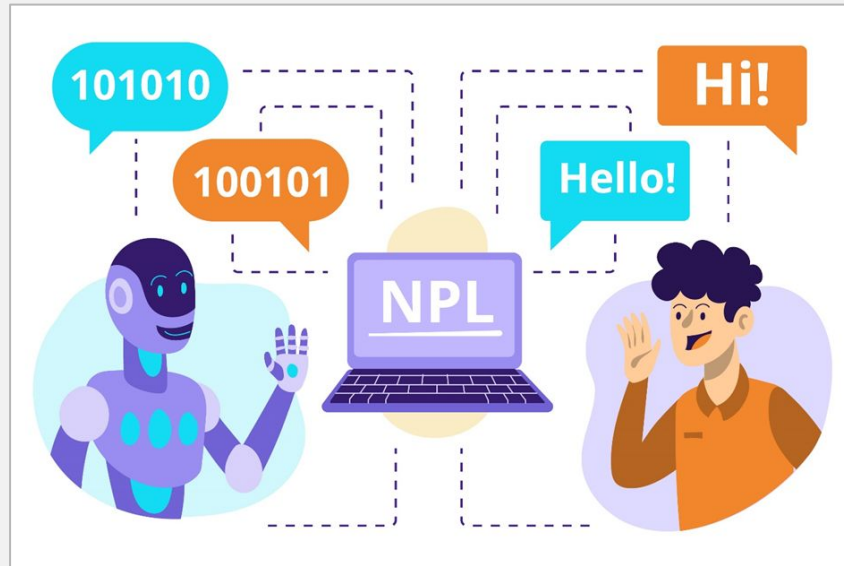


Use Cases for Text

When do we work with text data?

Natural Language Processing (NLP) is the field of computer science and artificial intelligence that uses principles of statistics and probability to **translate human language into a form computers can use** to solve problems.

Let's see a few examples.



Common Use Cases

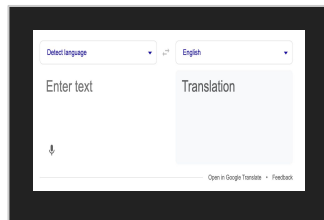
1

Searching the
web



2

Translation



3

Summarization
of documents



4

Identifying
entities



Representing Text with Numbers

How Computers See the World

Computers understand the world in numbers. In order to use text and image data we have to first represent it in a way that computers can interpret.

In text, we do this with **word** or **text embeddings**, which are numerical representations of words or groups of words.

Text for humans

I'm learning NLP



Numbers for computers

[0, 1, 0, 0, 1, 0]
[1, 0, 0, 0, 1, 0]
[0, 0, 0, 1, 0, 1]

NLP General Workflow

Raw Text

We start with a collection of raw text.

This group of documents is your **corpus**.

A **document** is a piece of text that represents one unit of analysis from the corpus.

Data Cleaning

Steps vary by corpus. Common actions include:

- **Removing duplicates** to speed up models and reduce cost.
- **Removing identifying information** such as names for privacy.
- **Removing stop words** (common words like **a**, **the**, etc.).
- Simplifying text using methods such as **stemming/lemmatization**.

Pre-processing for computers

Tokens or **N-grams** are the unit of analysis. Often these are **words or pieces of words** or even characters. You must **use the same tokenization method your model used for training** (BERT Tokenizer → BERT model). Units are then **converted into numbers** the computer can use.

Model

Typically, you will use a **pre-trained model** to that you apply to your text. These models are built vast amounts of training data (think all of Facebook, all of GitHub, all books, all of reddit, all scientific journal articles, the full web). You can also **build a model from scratch** but this **is very, very rare**.

Application

This is where you use your text for some purpose such as **summarization, translation, classification**.



NLP General Workflow



Raw Text

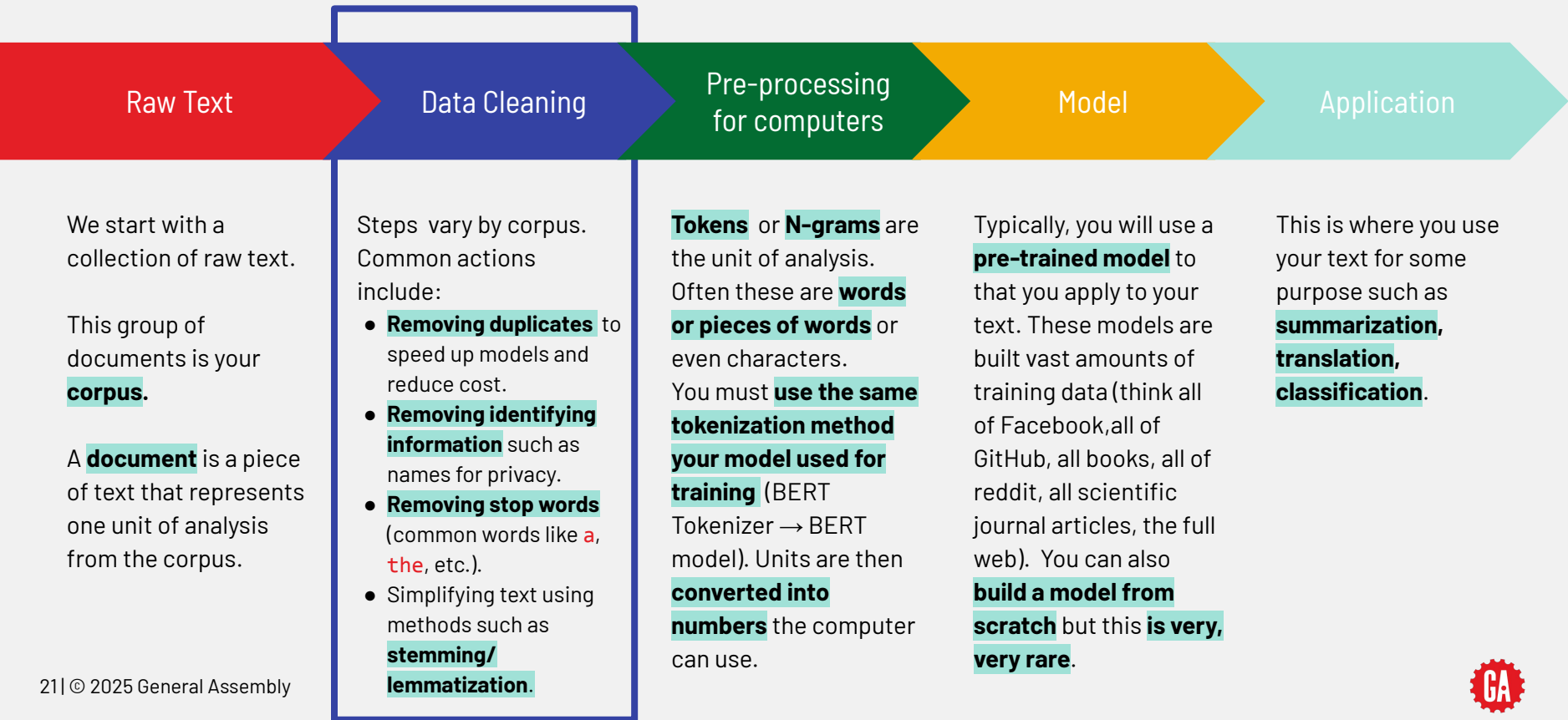
Raw text

I love learning about text.

We start with a collection of raw text.
This group of documents is your **corpus**.

A **document** is a piece of text that represents one unit of analysis from the corpus. It could be a sentence, a paragraph, a social media post, or any other piece of text.

NLP General Workflow



NLP General Workflow



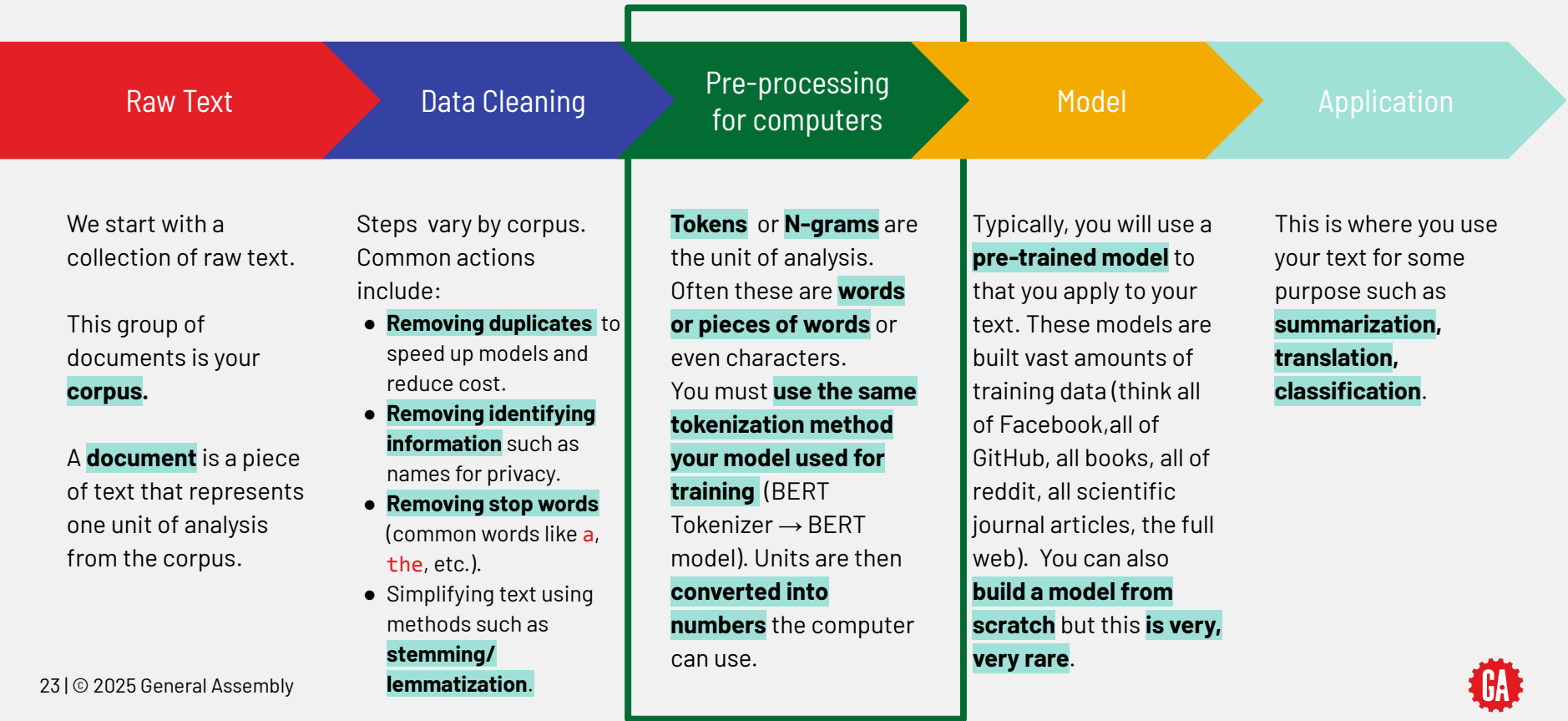
Data Cleaning

Steps vary by corpus. Common actions include:

- **Removing duplicates** to speed up models and reduce cost.
- **Removing identifying information** such as names for privacy.

Older methods such as **removing stop words** (common words like a, the, etc.) or simplifying text using methods such as **stemming/ lemmatization**, which shortens words to their root meaning, aren't used with most modern models.

NLP General Workflow



NLP General Workflow

Pre-processing for computers

Tokens or **N-grams** are the unit of analysis. Often these are **words or pieces of words** or even characters.

You must **use the same tokenization method your model used for training**.

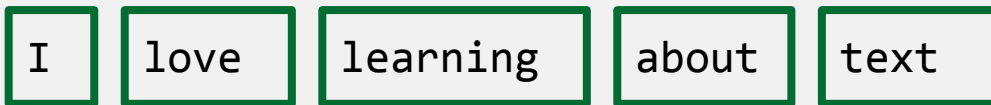
These units are then **converted into numbers** the computer can use. We will see this in action later in the module.

Raw text

I love learning about text.

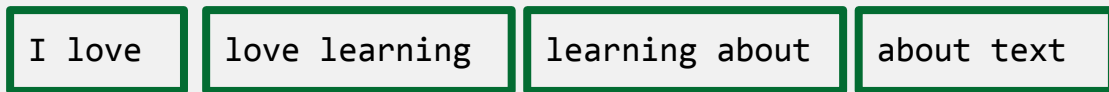


N-grams



or

N-grams (bi-grams)





Trivia

What do we call a the collection of all our documents for analysis?

A

N-gram

B

Pre-trained model

C

Corpus

D

Tokens



Trivia

Which of the following could be N-grams or tokens for the word “learning” from the phrase “Learning is fun”?

Raw text: **Learning is fun**

A

learning

B

learn

C

learning is

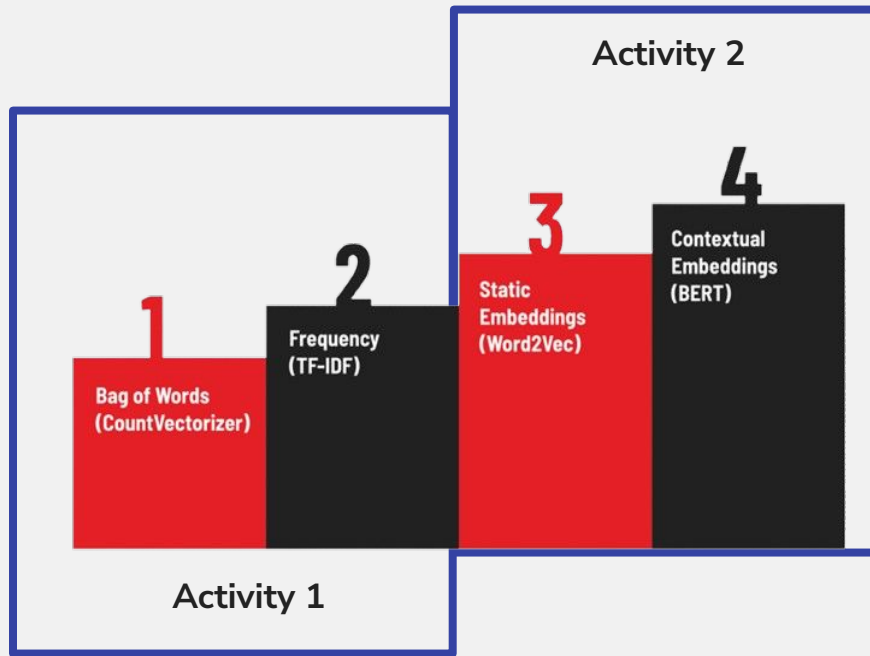
D

all the above

Exploring various text representation methods

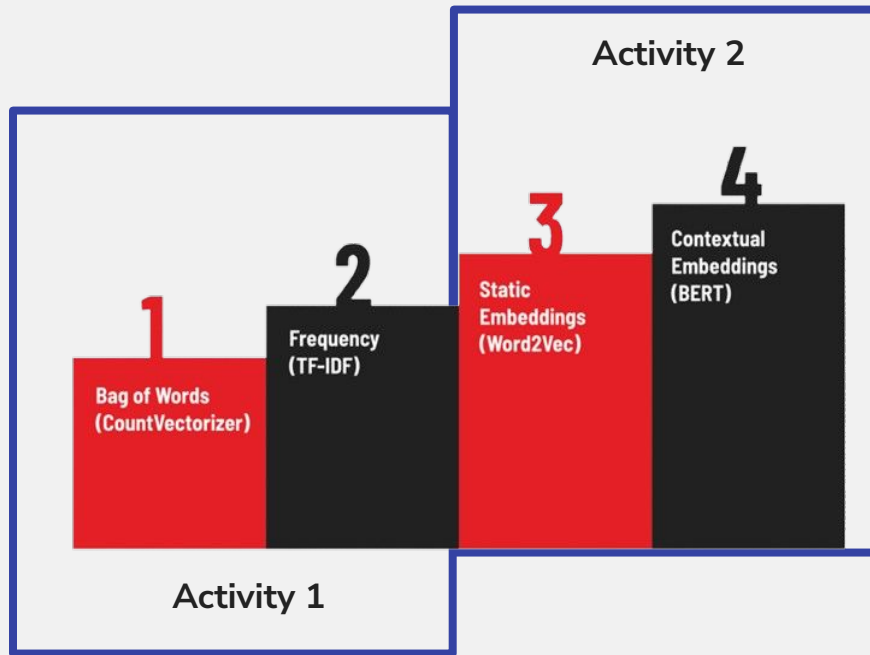
During the remainder of this module
you will explore 4 different ways to
represent text numerically.

They become progressively more
complex and capture more and more
context.



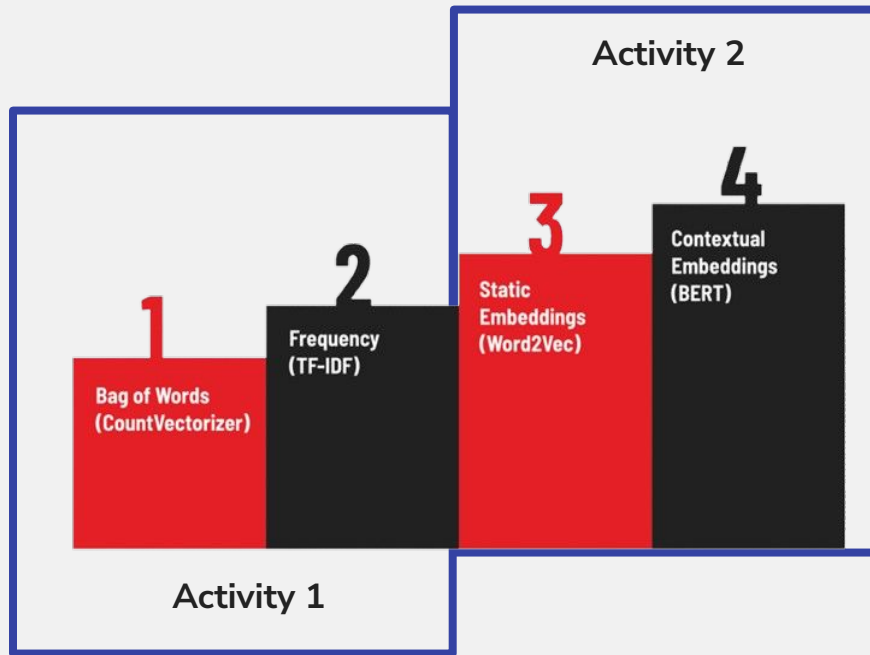
Exploring various text representation methods

Even the simplest method (CountVectorizer) can be extremely effective at representing text, particularly when paired with a classification model.

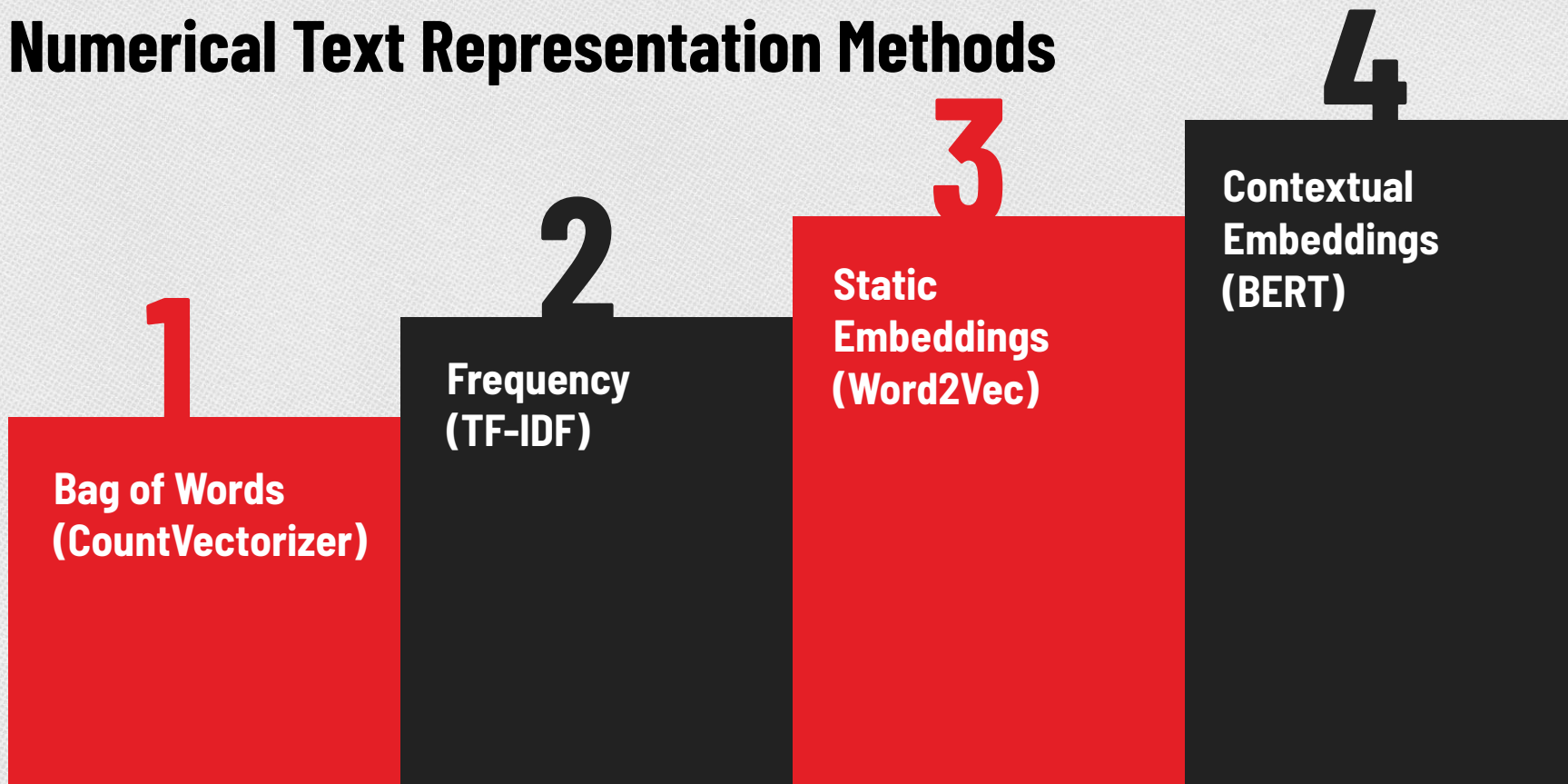


Exploring various text representation methods

When deciding which method to use, **consider starting with the simplest option and moving to progressively more complex** (and computationally expensive) only if you can't accomplish your goal with a simpler method.



Numerical Text Representation Methods



Numerical Text Representation Methods

1

Bag of Words (CountVectorizer)

Counts how frequently a word is used.

CountVectorizer

Let's look at one Bag of Words method:
CountVectorizer.

Suppose we have three short social media posts:

1. I love learning about text.
2. Text models are fun.
3. I love learning at GA. Learning is fun!



Bag of Words

Bag of Words captures the **frequency** of words but not the order.



Bag of Words

CountVectorizer creates **a column** for **every n-gram** that is used in our corpus.

n-gram = in this scenario, it is a word.

corpus = the collection of documents.

Documents

1. I love learning about text.
2. Text models are fun.
3. I love learning at GA.
Learning is fun!



	I	love	learning	about	text	models	are	fun	at	GA	is
I love learning about text											
Text models are fun											
I love learning at GA. Learning is fun!											

Bag of Words

Then we **count how often words or n-grams were used in each document** (or post in this case) and organize them into a table like this one.

1. I love learning about text.
2. Text models are fun.
3. I love learning at GA.
Learning is fun!



	I	love	learning	about	text	models	are	fun	at	GA	is
I love learning about text	1	1	1	1	1	0	0	0	0	0	0
Text models are fun	0	0	0	0	1	1	1	1	0	0	0
I love learning at GA. Learning is fun!	1	1	2	0	0	0	0	1	1	1	1

Bag of Words

Here we use a single word per column.
You can also create **bi-grams**, which
have **two words** per column.

1. I love learning about text.
2. Text models are fun.
3. I love learning at GA.
Learning is fun!



	I	love	learning	about	text	models	are	fun	at	GA	is
I love learning about text	1	1	1	1	1	0	0	0	0	0	0
Text models are fun	0	0	0	0	1	1	1	1	0	0	0
I love learning at GA. Learning is fun!	1	1	2	0	0	0	0	1	1	1	1

Numerical Text Representation Methods

1

Bag of Words (CountVectorizer)

Counts how frequently a word is used.

2

Frequency (TF-IDF)

Builds on CountVectorizer to account for how often words are used in your data.

Count Vectorizer Drawback

The count vectorizer is intuitive - but is it optimal? When using these counts in a model, we're making the implicit assumption that, for a term in our vocabulary:

more occurrences = more important

is this necessarily true?

TF-IDF

TF-IDF combines two metrics:

Term Frequency (TF)

How often a word appears in a document.

Inverse Document Frequency (IDF)

How unique that word is across all documents.

This helps identify words that are both frequent and meaningful in specific documents.
The full equation is [available here](#).

The TF-IDF Vectorizer

The logic behind this is we want to determine which terms in our corpus are the most **informative**.

A term is informative if it appears at least once in a document (high **tf**) but in not too many documents (low **df**), thus resulting in a high **TF-IDF**.

(Aside: this vectorization process is unsupervised - we aren't using our *y*-variable at all here! Which terms *actually* predict *y* are unknown to us at this time.)

TF-IDF

Term Frequency - Inverse Document Frequency (TF-IDF) takes into account **how often** a word is used with a **given document** (or post in this example) **and across our full corpus.**

The higher the TF-IDF, the more informative the word.

1. I love learning about text.
2. Text models are fun.
3. I love learning at GA.
Learning is fun!



	about	are	at	fun	ga	is	learning	love	models	text
I love learning about text	0.60	0	0	0	0	0	0.46	0.46	0	0.46
Text models are fun	0	0.56	0	0.43	0	0	0	0	0.56	0.43
I love learning at GA. Learning is fun!	0	0	0.39	0.29	0.39	0.39	0.60	0.29	0	0

TF-IDF

This is helpful for **removing** very rare **words that aren't meaningful** for our analysis as well as very common words that are used in every document many times.

These very common words are also called **stop words**.



Text Frequency Jupyter Notebook (text-frequency.ipynb)

Let's see it for ourselves!

- Open **text-frequency.ipynb**
- You will walk through **CountVectorizer** and **TF-IDF** to see how they work.



Text Frequency Jupyter Notebook (text-frequency.ipynb)

- ☐ This will serve as proxy for sentiment analysis.
- ☐ Remember even these simple methods can be surprisingly effective.



- ☐ Create a frequency word vector with CountVectorizer
- ☐ Compare methods to modify n-grams with stop words and length
- ☐ Explore the impact of incorporating TF-IDF
- ☐ Apply methods to predict customer rating based on review text (a proxy for sentiment analysis)

Frequency Methods - Bag of Words (Countvectorizer) with TF-IDF

PROS

Represents text numerically.

Fast and inexpensive.

Tailored to the data being used.

Capture how frequently words are used in our documents.

Can be used to approximate sentiment and in predictive models as features.

CONS

Doesn't account for different definitions of words

Can't be used for word comparisons

Doesn't account for the order of words in a phrase or sentence

Can miss important information, such as negations. For example:

I'm happy!/I'm **not** happy!

End of Part 1

Text Embeddings

Embeddings

We have now seen **2 methods for representing text:**

1. CountVectorizer

2. TF-IDF

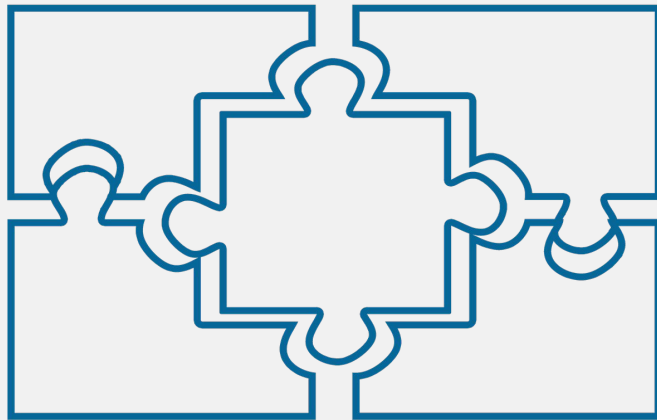
These methods are **surprisingly useful** despite having **no context** to how the word is used in the document.



Embeddings

But sometimes we do **need the context** of how the word is used and even where in a sentence that word falls.

These more complex numerical representations of text are called **embeddings**.



Embeddings use vectors to represent text

Embeddings use a **vector of numbers** to represent a token.

These additional numbers allow you to **capture more meaning about the word.**

- What is the definition of the word?
- How is it used in this context?
- How does that change the meaning of text?

Countvectorizer and TF-IDF use a **single value** to represent a token/n-gram.

Embeddings use a **vector of numbers** to represent a token. The dimensions (features) of the vectors vary from model to model.

Embeddings uniquely represent text

Let's look at a single word: **interest**. It has multiple meanings, depending on the context when it is used.

Countvectorizer and TF-IDF

Embedding

Single representation

#countvectorizer
interest = 1

#TF-IDF
interest = 0.23

Interest (noun)

Definition 1: the state of wanting to know or learn about something or someone.

Definition 2: money paid regularly at a particular rate for the use of money lent, or for delaying the repayment of a debt.

Unique representation

interest = [0.43, 0.28, 0.002,
0.03, 0.02, 0.14,
0.05, 0.10, 0.14]

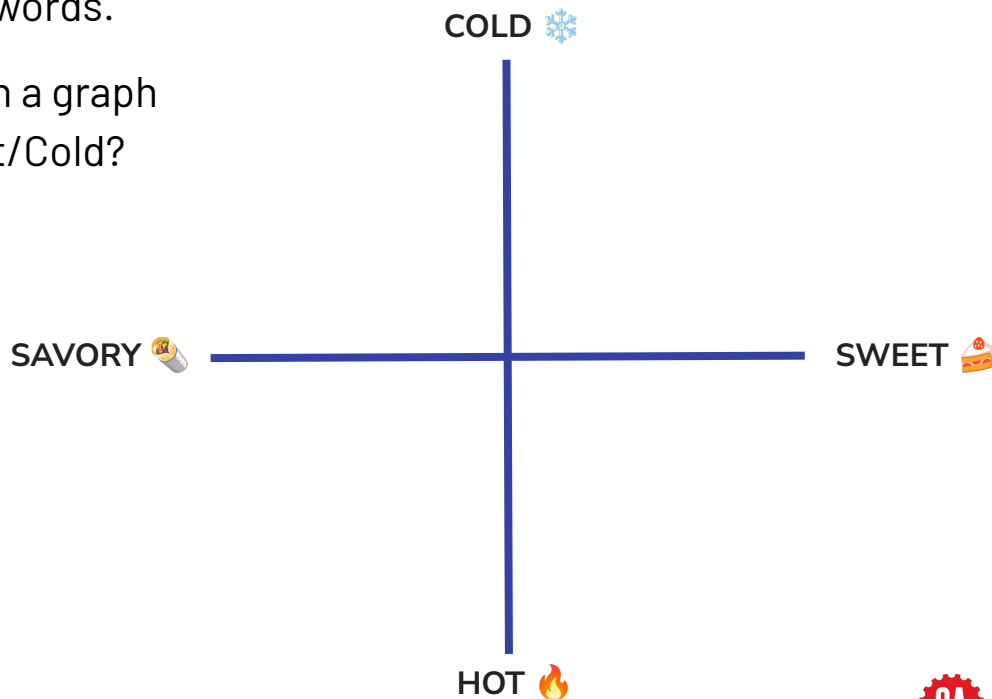
interest = [0.03, 0.15, 0.002,
0.22, 0.04, 0.06,
0.14, 0.01, 0.08]

Where do the vector numbers come from?

Let's say you had a group of food words.

How could you represent those on a graph comparing Sweet/Savory and Hot/Cold?

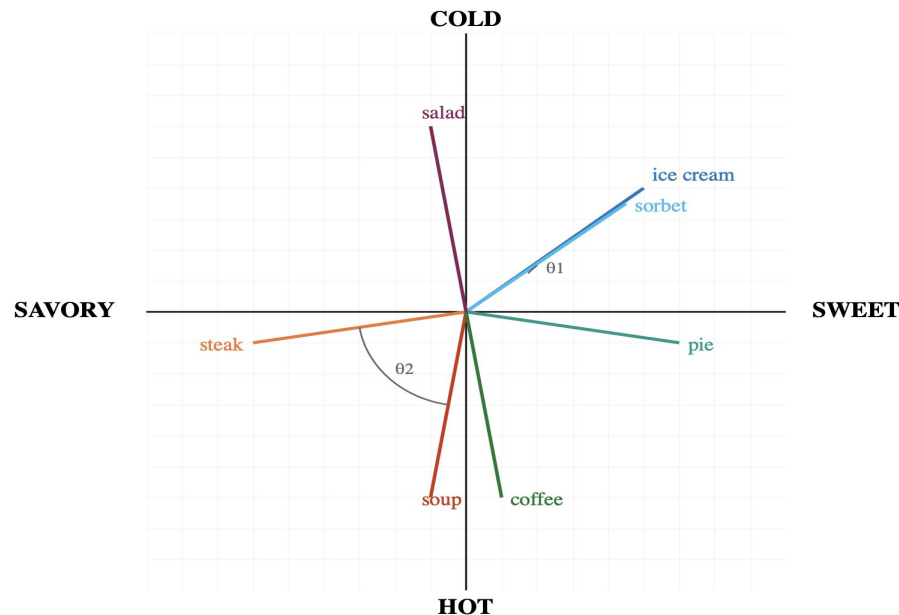
- ☐ Steak
- ☐ Soup
- ☐ Salad
- ☐ Coffee
- ☐ Pie
- ☐ Ice Cream
- ☐ Sorbet



Where do the vector numbers come from?

We can do this on multiple dimensions and get a numerical representation based on where words are plotted.

This becomes the basis for our vectors. The exact formula varies by model.



Numerical Text Representation Methods

1

Bag of Words (CountVectorizer)

Counts how frequently a word is used.

2

Frequency (TF-IDF)

Builds on CountVectorizer to account for how often words are used in your data.

3

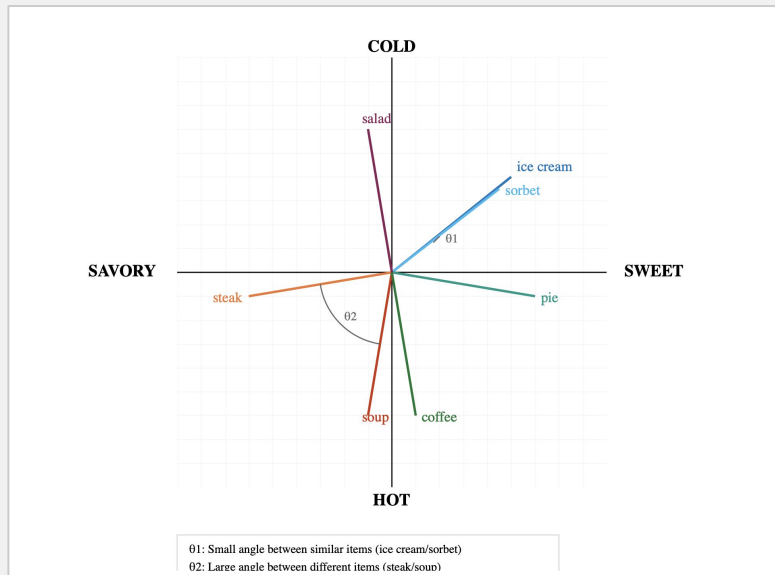
Static Embeddings (Word2Vec)

An embedding method that accounts for how a word is used.

Static Embeddings with Word2Vec

Text embeddings aim to capture the meaning of words by assigning them vectors, where similar words have similar vectors.

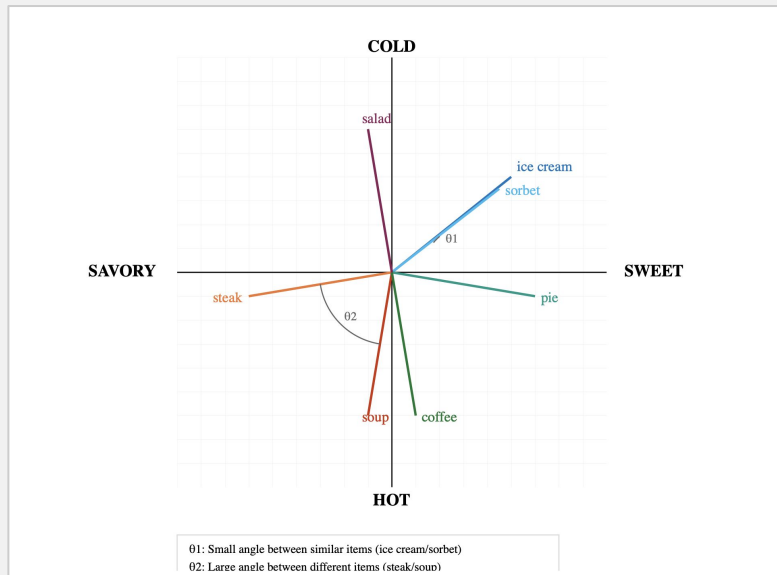
These vectors reflect the meanings or contexts in which the words are used. A given word will have a set vector regardless of the words around it. This is a static embedding.



Static Embeddings with Word2Vec

Word2Vec is **one common example** of **this type of embedding** that was developed by Google to help with translation tasks.

It makes it possible to capture different definitions/uses of words and compare words to each other.

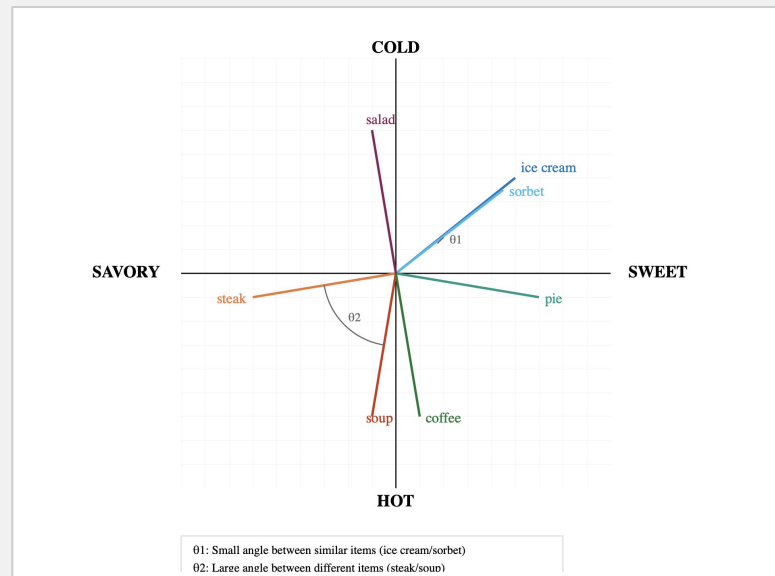


Word similarities with Word2Vec

Notice that angles between words can be used to see how similar two items are to each other.

Ice cream and **sorbet** are **very similar** to each other, while **salad** and **coffee** are **very different** from one another.

In mathematical terms, we can measure these differences using a **cosine similarity**.

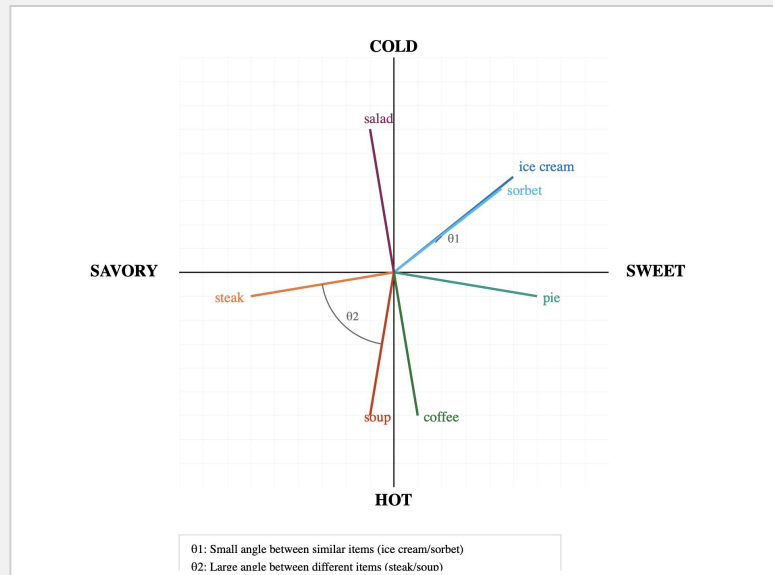


Word similarities with Word2Vec

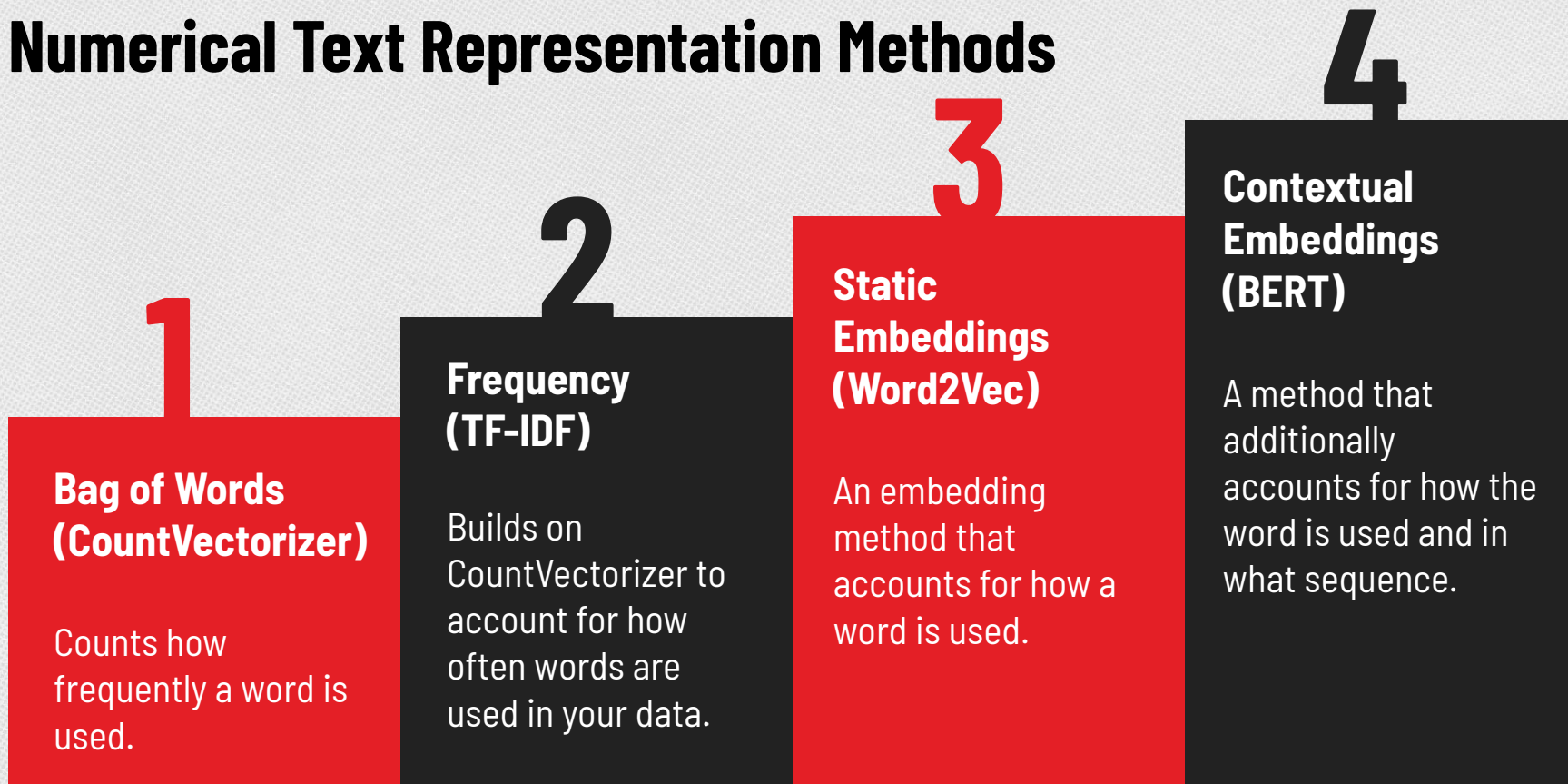
Cosine Similarity ranges in values from **-1 to 1**.

- **1** is an angle (θ) that is close to 0° or perfectly aligned (ice cream and sorbet are close in our example).
- **0** indicates that vectors are orthogonal i.e., completely dissimilar (coffee and steak in our example).
- **-1** is 180° and indicates completely opposite (coffee and salad in our example).

You can walk through the equation [here](#).



Numerical Text Representation Methods



Consider the following two examples

□ I'm **happy**!

□ I'm **not happy**!

→ "The **supplier agreed to pay** the manufacturer"

→ "The **manufacturer agreed to pay** the supplier"

In both cases **the order of the words impacts the meaning significantly**. Static embeddings, like Word2Vec, can't capture these changes.

You will need to a more advanced model to determine the full meaning of the words given the context of the words that were used around it.

Contextual Embeddings with BERT

BERT was designed to capture the impact of word order. It is used **to create contextual embeddings**.

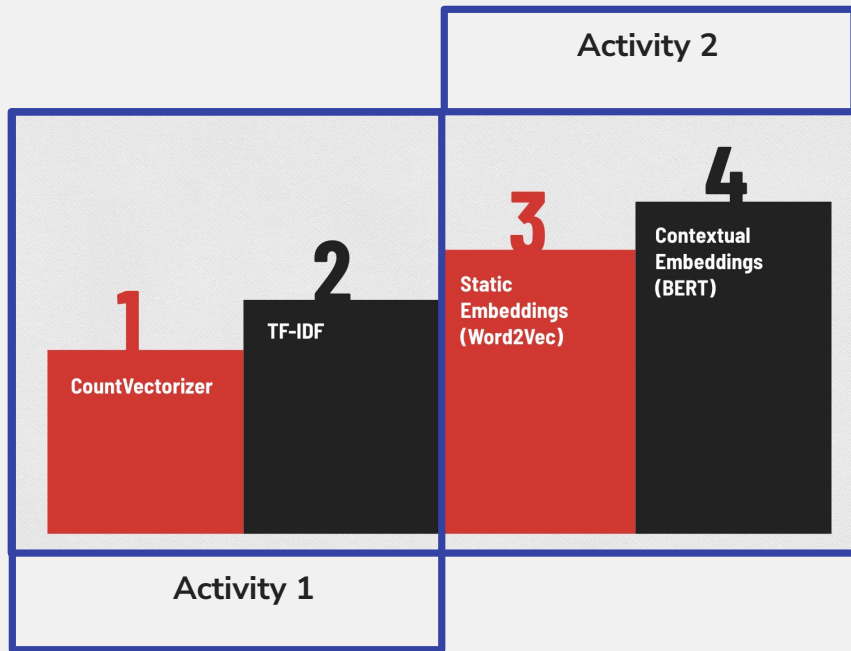
BERT is a famous language model developed by Google that uses transformer encoder architecture. We will explore this architecture in more detail in future modules. Let's compare the embeddings produced by Word2Vec and BERT in the next activity.



Exploring various text representation methods

In our next activity we compare static contextual embeddings (word2vec) with contextual embeddings (BERT).

We will use 2 models that represent each of these (Word2Vec and BERT) but there are other options to explore in the future.



Embeddings

- In this notebook, you will compare two embedding methods, **Word2Vec** and **BERT**.
- These are more complicated models that may be necessary for a given task (such as complex sentiment analysis).
- You will focus on how each model represents text as vectors.



Embeddings

Let's see it for ourselves!

→ Open **embeddings.ipynb**



- ☐ What's the difference between static and contextual embeddings?
- ☐ Why might you choose static word2vec embeddings over contextual ones?

Static Embeddings - Word2Vec

PROS

- Represents text numerically.
- Fast and inexpensive.
- Tailored to the data being used.
- Accounts for different word definitions.
- Can be used for word comparisons.
- Helpful for many tasks including translation, sentiment analysis.

CONS

- Doesn't account for the order of words in a phrase or sentence.
- Can miss important information, such as negations. For example:
 - I'm happy! / I'm **not** happy!
- Doesn't work well on small datasets.

Contextual Embeddings - BERT

PROS

Represents text numerically.

Fast and inexpensive.

Tailored to the data being used.

Accounts for different definitions of words.

Can be used for word comparisons.

Account for the order of words in a phrase or sentences.

Pre-trained on vast amounts of data, so works ok on small datasets similar to the training data.

Captures important information, such as negations.

For example:

☐ I'm happy! / I'm **not** happy!

CONS

Can be slow and expensive to run on large datasets.

Extra complexity may not be required when a simpler, faster tool can accomplish the task equally well.

As a pre-trained model, it will contain biases that may be difficult to detect.

Impact of Method Selection

Real-World Impacts

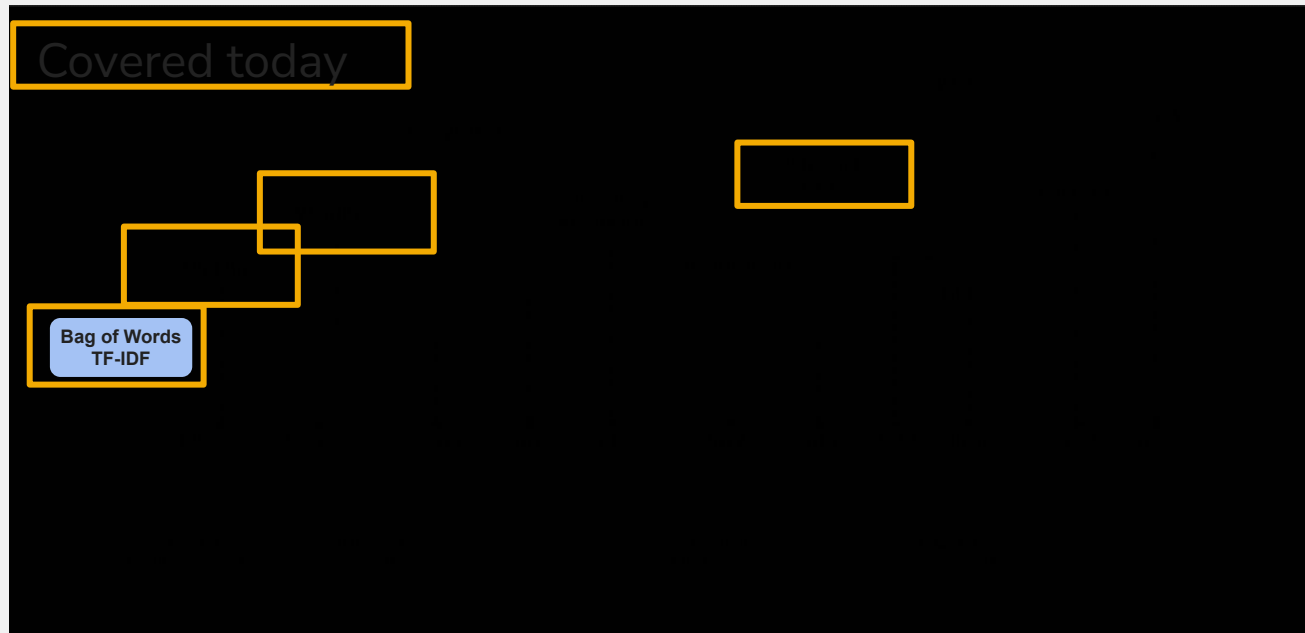
What are the impacts of how you represent your text numerically?

- ☐ Outline the +/- of the methods we discussed.
 - ☐ Describe when you may choose a simpler method over a complex contextual one.
 - ☐ Consider the role of data quality and setup choices (such as ,stop word removal) on your final results.
- ☐ How do you decide which model to use?
 - ☐ What is the impact on cost? Processing time?
 - ☐ Are any methods more prone to bias than others?

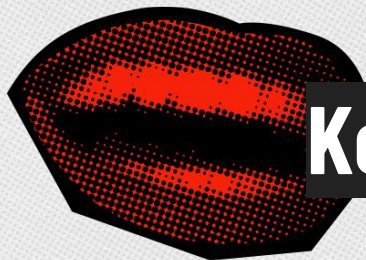
Bring It Home

Timeline of the models you used today

This is a rapidly developing space with new models coming out frequently, each new model building on the foundations of the ones that came before.



Modified from Source: Wang, Z., Chu, Z., Doan, T. V., Ni, S., Yang, M., & Zhang, W. (2024). History, development, and principles of large language models: an introductory survey. *AI and Ethics*, 1-17. ([preprint](#))



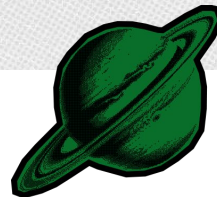
Key takeaways



There are a variety of ways to represent text.



You should use the same method that was used to train your model.



If you aren't using a model, use the simplest method that accomplishes the task.



Additional Resources

DIGGING DEEPER

Bag-of-Words

- <https://www.ibm.com/think/topics/bag-of-words>

Word2Vec

- <https://arxiv.org/abs/1301.3781>

BERT

- <https://arxiv.org/abs/1810.04805>

